

✓ QUESTION 02

```
import math

# Minimax function
def minimax(depth, nodeIndex, isMax, values, maxDepth):

    # Base case: leaf node
    if depth == maxDepth:
        return values[nodeIndex]

    if isMax:
        return max(
            minimax(depth + 1, nodeIndex * 2, False, values, maxDepth),
            minimax(depth + 1, nodeIndex * 2 + 1, False, values, maxDepth)
        )
    else:
        return min(
            minimax(depth + 1, nodeIndex * 2, True, values, maxDepth),
            minimax(depth + 1, nodeIndex * 2 + 1, True, values, maxDepth)
        )

# Example leaf nodes (from your board)
values = [-4, -5, 2, 3, 4, 5, 6, 7]

# Tree depth = log2(n)
depth = int(math.log2(len(values)))

result = minimax(0, 0, True, values, depth)

print("Optimal Value (Minimax):", result)

Optimal Value (Minimax): 5
```

✓ QUESTION 01

```
def minimax(node, is_max, graph, values):

    # If leaf node → return its value
    if node not in graph:
        return values[node]

    children = graph[node]
```

```

    if is_max:
        best = float("-inf")
        for child in children:
            best = max(best, minimax(child, False, graph, values))
        return best

    else:
        best = float("inf")
        for child in children:
            best = min(best, minimax(child, True, graph, values))
        return best

# ----- GRAPH (FULL 3 LEVEL TREE) -----
graph = {
    "A": ["B", "C"],

    "B": ["D", "E"],
    "C": ["F", "G"],

    "D": ["H", "I"],
    "E": ["J", "K"],
    "F": ["L", "M"],
    "G": ["N", "O"]
}

# ----- LEAF VALUES (LEVEL 3) -----
values = {
    "H": 3,
    "I": 5,
    "J": -2,
    "K": 4,
    "L": 6,
    "M": 1,
    "N": -3,
    "O": 2
}

# ----- RUN MINIMAX -----
result = minimax("A", True, graph, values)

print("Result:", result)

```

Result: 4

✓ Question 03

#QUESTIONS IN CODE

```
# Q3: Create Important Questions from Code (at least 20)

# Here are 20 strong, exam-ready questions based on YOUR code:

# 🌟 Conceptual + Code-Based Questions
# What is Minimax algorithm?
# What problem does Minimax solve?
# What are MAX and MIN players?
# Why recursion is used in Minimax?
# What is a terminal (leaf) node?
# What is depth of a tree?
# What is nodeIndex used for?
# Why nodeIndex * 2 is used?
# Why nodeIndex * 2 + 1 is used?
# What is base condition in Minimax code?
# How does MAX choose values?
# How does MIN choose values?
# What is time complexity of Minimax?
# What is space complexity?
# What is a complete binary tree?
# Why  $\log_2(n)$  is used for depth?
# What happens if tree is not complete?
# Can Minimax work without recursion?
# Where is Minimax used in real life?
# What are limitations of Minimax?
```

✓ Question 04

SYNTAX OF MIN MAX ALGORITHM

```
def minimax(node, is_max, graph, values):
    # Assuming 'terminal' nodes are those not in the graph
    if node not in graph:
        return values[node]

    children = graph[node]

    if is_max:
        best = float('-inf')
        for child in children:
            best = max(best, minimax(child, False, graph, values))
        return best

    else:
```

```
best = float('inf')
for child in children:
    best = min(best, minimax(child, True, graph, values))
return best
```

TASK

✓ QUESTION 01

Start coding or [generate](#) with AI.